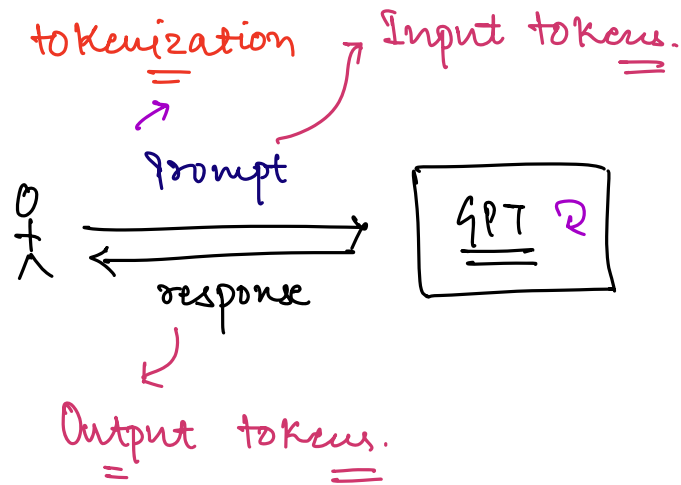
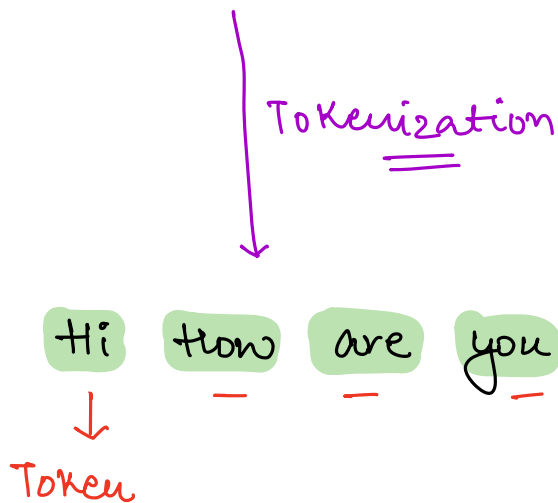




Tokenization

→ It is the process of converting text into smaller units called as tokens that LLM can process.

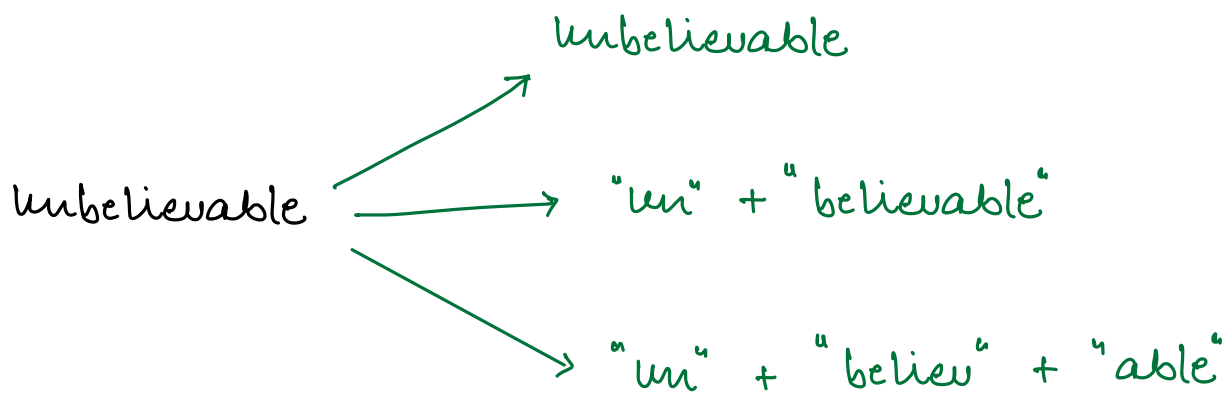
Hi, how are you?



In reality, token $\neq$ word

Rough Estimate

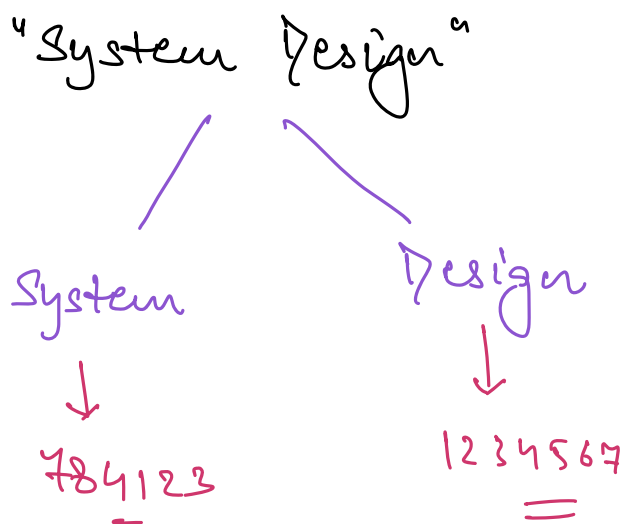
1 token $\approx$ 0.75 words.



Why do LLMs need Tokenization?

→ LLMs don't understand text.

↳ Only understand numbers.



LLM cost is measured by the no. of tokens it has consumed in the query.

Cost of output tokens > Cost of input tokens.

Prompt Optimization Techniques.

Prompt optimization techniques:

1. Trim instructions – remove redundant or overly verbose phrasing
2. Compress examples – use fewer, more targeted few-shot examples
3. Avoid repetition – do not restate the same instruction multiple ways
4. Shorten system prompts – keep persona and rules concise
5. Truncate or summarize long context instead of including it whole

UNOPTIMIZED (verbose, repetitive):

"I would like you to please carefully analyze the following customer review and determine, to the best of your ability, whether the sentiment expressed by the customer in this review is positive, negative, or neutral in nature. Please make sure to think about this carefully and provide an accurate classification. Here is the review that I would like you to analyze: 'Great product, fast shipping!'"

≈ 75 tokens

OPTIMIZED:

"Classify the sentiment of this review as positive, negative, or neutral: 'Great product, fast shipping!'"

≈ 20 tokens

Caching

- helps us to avoid making expensive AI calls.
- saves cost & latency.

Batch Processing

- Not every AI workload needs an immediate response. For such tasks, we can execute the queries in batches / groups.
- If we make requests one-by-one sequentially and waiting for the response for each request before making the next request, then it is going to be time consuming & expensive.
- ⇒ Batch Processing allows us to submit a large no. of requests together, these requests can be processed asynchronously & it will provide us a big optimization in terms of cost & latency.

Real-time processing:

Request 1 → wait → response 1 → Request 2 → wait → response 2 → ...

Fast turnaround per request, full price, needed for user-facing features

Batch processing:

Submit all requests together → provider processes asynchronously →

retrieve all results once complete (often minutes to hours later)

Slower overall turnaround, reduced price, ideal for non-urgent bulk work

Model Selection

Model selection guide:

Simple, well-defined tasks (classification, extraction, short answers)

→ smaller, cheaper, faster model

Complex reasoning, nuanced writing, ambiguous or high-stakes tasks

→ larger, more capable model

Rule of thumb: start with the cheapest model that could plausibly work, test it against real examples, upgrade only if quality is insufficient.

— * —